

Practical : 12	Write a Program for the Travelling Salesman Problem.
Date : 12/08/2026	

Objective :- To implement a solution to the Travelling Salesman Problem (TSP) in C++ using the Held-Karp dynamic-programming algorithm to find the minimum-cost Hamiltonian tour, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;

const int INF = 1e9;

// ----- TSP - Held-Karp Dynamic Programming,  $O(n^2 * 2^n)$  -----
int main() {
    int n = 4;
    vector<vector<int>>> dist = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    int VISITED_ALL = (1 << n) - 1;
    // dp[mask][i] = min cost to have visited 'mask' set of cities, ending at city i
    vector<vector<int>>> dp(1 << n, vector<int>(n, INF));
    vector<vector<int>>> parent(1 << n, vector<int>(n, -1));
    dp[1][0] = 0; // start at city 0

    for (int mask = 1; mask <= VISITED_ALL; mask++) {
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue; // already visited
                int nextMask = mask | (1 << v);
                int newCost = dp[mask][u] + dist[u][v];
                if (newCost < dp[nextMask][v]) {
                    dp[nextMask][v] = newCost;
                    parent[nextMask][v] = u;
                }
            }
        }
    }

    int bestCost = INF, bestEnd = -1;
```

```
for (int u = 1; u < n; u++) {
    if (dp[VISITED_ALL][u] == INF) continue;
    int cost = dp[VISITED_ALL][u] + dist[u][0];
    if (cost < bestCost) { bestCost = cost; bestEnd = u; }
}

// reconstruct optimal route from parent pointers
vector<int> route;
int mask = VISITED_ALL, cur = bestEnd;
vector<int> tmp;
while (cur != -1) {
    tmp.push_back(cur);
    int pu = parent[mask][cur];
    mask ^= (1 << cur);
    cur = pu;
}
for (int i = tmp.size() - 1; i >= 0; i--) route.push_back(tmp[i]);
route.push_back(0);

cout << "Optimal Tour: ";
for (size_t i = 0; i < route.size(); i++) {
    cout << route[i];
    if (i + 1 < route.size()) cout << " -> ";
}
cout << "\nMinimum Tour Cost: " << bestCost << endl;
return 0;
}
```

Output :-

Optimal Tour: 0 -> 2 -> 3 -> 1 -> 0
Minimum Tour Cost: 80

Analysis:-

Time & Space Complexity (n = number of cities):

Method	Time Complexity	Space Complexity
Brute Force (permutations)	$O(n!)$	$O(n)$
Held-Karp Dynamic Programming	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$

The Travelling Salesman Problem asks for the minimum-cost tour that visits every city exactly once and returns to the start. A brute-force search over all $(n-1)!$ possible orderings is correct but grows factorially and is impractical beyond about 10-12 cities. The Held-Karp dynamic-programming approach instead represents state as (subset of visited cities, current city) using a bitmask, so $dp[mask][i]$ holds the minimum cost to have visited exactly the cities in mask and end at city i . Since there are 2^n subsets and n possible end cities, and each state transitions to at most n next cities, the total time is $O(n^2 \cdot 2^n)$ and the space to store the DP table is $O(n \cdot 2^n)$ — an exponential improvement over brute force's $O(n!)$, though still exponential overall, which is expected since TSP is NP-hard. For the 4-city sample graph above, the DP explores all $2^4 = 16$ subsets and correctly finds the optimal tour $0 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$ with total cost 80, which matches the brute-force optimum for this small instance and can be verified directly: $15 + 30 + 25 + 10 = 80$.

Conclusion :- The Travelling Salesman Problem was successfully implemented in C++ using the Held-Karp

bitmask dynamic-programming technique. The program correctly found the optimal tour and minimum cost for the sample 4-city graph, confirming the $O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ space behaviour of the DP approach, a substantial improvement over the $O(n!)$ cost of brute-force enumeration. Because both bounds are exponential in n , Held-Karp remains practical only for a few dozen cities at most; for larger instances, heuristic and approximation methods such as nearest-neighbour, 2-opt, or genetic algorithms are used to find near-optimal tours in polynomial time.